Словарь терминов: RL, генерация и оценка

Термины, о которых шла речь в обсуждениях 07–08.11.2026 (устройство GRPO и PPO, генерация на V100, проверка награды, оценка SFT). Для каждого: что это, чем отличается от похожих, где возник у нас и где применяется. Подробные разборы — `docs/subtleties.md`, решения — `docs/decisions.md`, источники — `docs/references.md`.

> **Как читать карточку.** Что это — определение. Отличие — с чем легко спутать и в чём разница. Где возник — в каком месте нашего обсуждения появился термин. Где применяется — в каких работах и случаях он встречается и как используется у нас.

1. Организация обучения — группа, шаг, эпоха; μ; абляция; сид
2. Сигнал обучения — σ задания; преимущество; нормировки GRPO / Dr. GRPO / Lite PPO; критик; досэмплирование DAPO
3. Чьи данные и как их учитывать — on-/off-policy; importance sampling; ratio; клиппинг PPO; усреднение по токенам и ответам
4. KL и опорная модель — KL; k1, k2, k3; β фиксированный и адаптивный; KL к предыдущей политике; KL в VAE; опорная модель; LoRA и слияние
5. Оптимизатор и точность чисел — скорость обучения и размер шага; AdamW; weight decay; отсечение градиента; масштабирование loss; float16 / bfloat16; compute capability
6. Генерация — температура, top-k, top-p; generation_config; teacher forcing; vLLM; кеш KV; prefill и decode; PagedAttention; continuous batching; ядра внимания
7. Оценка — eval loss; перплексия; ROC AUC и доля верно упорядоченных пар; LLM-судья; distinct-n; забывание; выбор чекпоинта
8. Сводные сравнения — масштабирование и нормировка; размер шага и скорость обучения; три «клиппинга»; три KL

1. Организация обучения

Группа, шаг, эпоха group / step / epoch

Что это. **Группа** — несколько ответов модели на одно и то же задание (у нас 8). **Шаг** — одно обновление весов оптимизатором: собрали пакет из нескольких групп (у нас 8 заданий × 8 ответов = 64 эпизода), посчитали loss, сделали `optimizer.step()` один раз. **Эпоха** — один проход по всем заданиям обучающей выборки.

Отличие. Группа — единица сравнения (ответ лучше или хуже соседей), шаг — единица обновления, эпоха — единица охвата данных. В SFT «шаг» — тоже одно обновление, но пакет там — готовые примеры, а не сгенерированные эпизоды.

Где возник. В схеме «8 × 8» и в вопросе «100 или 150 шагов». Расчёт: 274 темы в train, 4 задания с поиском и 4 без на шаг → около 68 шагов на эпоху; 150 шагов ≈ 2,2 эпохи, то есть каждое сочетание «тема × режим» модель видит примерно 2 раза.

Где применяется. В любом RL для языковых моделей (GRPO, PPO, RLOO). В статьях размер пакета обычно задают как «число заданий × число ответов на задание».

μ — число обновлений на один пакет PPO epochs / μ

Что это. Сколько раз оптимизатор проходит по одному и тому же собранному пакету эпизодов, прежде чем выбросить его и сгенерировать новый. μ = 1 — каждый эпизод используется один раз; μ = 2–4 — как в классическом PPO.

Отличие. Не путать с эпохой по данным: «эпоха PPO» (μ) — повтор одного пакета, эпоха по данным — проход по всем заданиям. При μ > 1 обучение становится off-policy (раздел 3), и нужен клиппинг.

Где возник. Вопрос 5: «GRPO увидит одно задание только один раз — не маловато ли?». μ = 2 — вариант, если генерация окажется самой дорогой частью шага: повторный проход по тем же 64 эпизодам дешевле новой генерации.

Где применяется. PPO (Schulman и др., 2017), GRPO в DeepSeekMath (там μ = 1 в основном варианте). Для честного сравнения GRPO и PPO у нас μ должно быть одинаковым.

Абляция ablation

Что это. Эксперимент, в котором из системы убирают (или меняют) одну часть, оставляя всё остальное как было, и смотрят, как изменился результат. Так узнают вклад именно этой части.

Отличие. От подбора гиперпараметров: подбор ищет лучшее значение, абляция отвечает на вопрос «нужна ли эта часть вообще и сколько она даёт». От сравнения двух систем: там отличаются многие части сразу, и нельзя сказать, какая дала разницу.

Где возник. Пункт 2 (4 задания с поиском + 4 без на шаг) и проверка награды: переключатели `relative`, `keys_gate`, `contradiction` оставлены в коде выключенными именно для абляций; варианты в `perturb_eval.VARIANTS` — каждый с одним изменением от базового.

Где применяется. Почти в каждой статье по ML — таблица «без компонента X». В дипломе: вклад компонентов награды, роль KL, роль поиска.

Сид random seed

Что это. Начальное значение генератора случайных чисел. Фиксирует порядок заданий, случайные токены при генерации, инициализацию LoRA.

Отличие. Разные сиды при одинаковых настройках — это повторы одного эксперимента; разброс между ними показывает, какая часть различий между методами — случайность.

Где возник. Предложение «3 сида × GRPO и PPO»: в RL результат одного запуска сильно зависит от случая, и разница GRPO/PPO на одном сиде может быть шумом.

Где применяется. Отчёты RL обычно дают среднее и разброс по 3–5 сидам (Henderson и др., «Deep RL that Matters»).

2. Сигнал обучения

σ — разброс награды внутри задания

Что это. Стандартное отклонение награды между разными ответами на одно и то же задание (на одну тему), усреднённое по заданиям. У нас σ ≈ 0,07–0,08.

Отличие. От разброса награды по всей выборке: тот включает разницу между темами (одна тема легче другой), а сравнение в GRPO идёт только внутри задания. Поэтому для «насколько велик эффект» нужен именно внутренний разброс.

Где возник. Проверка награды на искажениях: пороги m = 0,25σ и ε = 0,1σ; все сдвиги в EXP-014/016 выражены в σ («выдуманная цитата −3σ»). Оценка «5 вызовов по 0,02 ≈ 1,4σ» — отсюда решение снизить цену вызова до 0,003.

Где применяется. Как единица «размера эффекта» (аналог d Коэна). В GRPO похожая величина — делитель преимущества.

Преимущество advantage, A

Что это. Насколько ответ лучше «ожидаемого» для этого задания: A = r − b, где b — базовая линия. Обучение повышает вероятность токенов ответов с A > 0 и понижает с A < 0.

Отличие. От награды: награда r — абсолютная оценка ответа; преимущество — относительная (по сравнению с базой). Без вычитания базы все ответы с положительной наградой подталкивались бы вверх, и сигнал был бы очень шумным.

Где возник. Устройство GRPO: база b — среднее награды группы (8 ответов на то же задание).

Где применяется. Все методы градиента политики: REINFORCE с базой, PPO (база — критик), GRPO и RLOO (база — среднее группы).

Нормировка преимущества: GRPO, Dr. GRPO, Lite PPO

Что это. Как привести преимущества к общему масштабу.

- **GRPO** (DeepSeekMath): A = (r − среднее группы) / std группы.
- **Dr. GRPO**: A = r − среднее группы, без деления.
- **Lite PPO** («Tricks or Traps»): A = (r − среднее группы) / std всего шага (пакета).
- **У нас**: A = (r − среднее группы) / max(std всего шага; 0,02).

Отличие. Деление на std группы растягивает любую группу до одного размера: группа из 8 почти равных ответов (разница — шум) получает такие же большие A, как группа с явным победителем. Деление на std шага даёт *общую* единицу для всех групп шага: вычитание среднего убирает уровень задания (лёгкая или трудная тема), а различия внутри группы сохраняют свой настоящий размер — группа с большим разбросом даёт больший сигнал. Нижняя граница 0,02 (≈ четверть σ) — чтобы при сходстве всех ответов шага сигнал не раздувался.

Где возник. Обсуждение с Codex и вопрос 3: «если сравниваем награды группы, почему делим на разброс шага — масштаб может быть разный?» Ответ: именно поэтому. Если бы у каждой группы был свой масштаб, сравнивать их сигналы было бы нельзя; одна единица на шаг сохраняет разницу «эта группа разошлась сильно, та — слабо». Делитель — один для всех 64 ответов шага, так что он меняет только общий размер шага обучения, но не направление и не соотношение групп.

Где применяется. GRPO — DeepSeekMath, DeepSeek-R1; Dr. GRPO — «Understanding R1-Zero-Like Training»; Lite PPO — «Tricks or Traps? A Deep Dive into RL for LLM Reasoning» (2025). Название Lite PPO обманчиво: критика там нет, это вариант GRPO.

Критик (функция ценности) critic / value function

Что это. Отдельная модель (или «голова»), которая по началу ответа предсказывает ожидаемую итоговую награду. В PPO она служит базой b для каждого токена.

Отличие. GRPO обходится без критика: база — среднее группы. Критик даёт базу на каждом токене, но его надо обучать (лишняя память и источник ошибок); среднее группы бесплатно, но требует нескольких ответов на задание.

Где возник. Выбор двух алгоритмов для сравнения: GRPO (без критика) и PPO (с критиком) — это главное различие между ними.

Где применяется. PPO, A2C, InstructGPT; в PPO для языковых моделей преимущество по токенам считают через GAE.

Группы с одинаковой наградой и досэмплирование DAPO dynamic sampling

Что это. Если все 8 ответов группы получили одну и ту же награду, все A = 0 — сигнала награды от группы нет. DAPO такие группы выбрасывает целиком (задание и все его ответы) и **досэмплирует**: берёт новые задания и генерирует для них группы, пока шаг не наполнится группами с разной наградой.

Отличие. Досэмплирование ≠ просто фильтр. Фильтр выбрасывает группы, и шаг становится меньше; досэмплирование возвращает размер шага, но стоит лишней генерации. Ни то, ни другое не генерирует новые ответы на *то же* задание.

Где возник. Вопрос 4 — вы поняли верно: старое задание с его ответами выбрасывается, берётся новое и для него генерируются ответы. У нас решено группы не выбрасывать: награда непрерывная, точная ничья 8 ответов почти невозможна; к тому же выброс убрал бы и KL на этих группах. Долю почти равных групп пишем в журнал.

Где применяется. DAPO (2025) — в задачах с наградой 0/1 (математика), где все 8 верных или все 8 неверных ответов — частый случай.

3. Чьи данные и как их учитывать

On-policy и off-policy

Что это. **On-policy** — учиться только на эпизодах, которые сгенерировала текущая модель, и выбрасывать их после одного обновления. **Off-policy** — учиться и на эпизодах, сгенерированных другой моделью: старой версией (PPO с μ > 1, буфер опыта) или вовсе чужой (DPO на готовых парах).

Отличие. On-policy проще и честнее (оценка градиента без поправок), но каждый эпизод используется один раз. Off-policy экономнее по данным, но требует поправки (ratio) и ограничения шага (клиппинг).

Где возник. Вопрос 5: «почему сравнение с PPO чище, если PPO использует другую политику?». «Чище» — не потому что PPO такой же, а потому что при одинаковом μ оба алгоритма одинаково on- или off-policy, и разница между ними сводится к базе (среднее группы против критика). Если GRPO взять с μ = 1, а PPO с μ = 4, в разницу примешалось бы ещё и «сколько раз использованы данные».

Где применяется. REINFORCE, GRPO с μ = 1 — on-policy; PPO с μ > 1, SAC, DQN, DPO — off-policy (в разной степени).

Importance sampling и отношение вероятностей ratio, ρ

Что это. Поправка, позволяющая оценивать «как было бы при новой модели» по данным старой: каждый токен взвешивается отношением $\rho = \pi_\theta(\text{токен} \mid \text{контекст}) / \pi_{old}(\text{токен} \mid \text{контекст})$, где π_θ — модель с текущими весами, π_{old} — модель, которая сгенерировала эпизод. В коде: `ratio = exp(logp_new - logp_old)`. Цель (без клиппинга): L = −среднее(ρ · A).

Отличие. Значение ρ и его градиент — разные вещи. При μ = 1 ρ = 1 в точке, но $\nabla\rho = \nabla \log \pi_\theta \neq 0$. Поэтому в коде `logp_old = logp.detach()`, а не константа 1: с константой loss не зависит от весов, градиент ноль, и модель не учится, хотя всё работает без ошибок.

Где возник. Оговорка Codex и вопрос 5 «что такое ratio, какая формула, как и где работает». Работает в функции loss шага обучения, на каждом токене ответа модели (токены среды не учитываются).

Где применяется. PPO, GRPO, TRPO — везде, где обновление может идти на чуть устаревших данных; при μ = 1 сводится к обычному градиенту политики $-A \cdot \nabla \log \pi$.

Клиппинг PPO clipped surrogate objective

Что это. Цель max(min(ρ·A, clip(ρ, 1−ε, 1+ε)·A)) с ε ≈ 0,2: когда вероятность токена уже изменилась больше чем на 20% относительно старой модели, градиент за этот токен обнуляется — дальше её не тянут.

Отличие. Ограничивает изменение *вероятностей*, а не размер градиента (это отсечение градиента, раздел 5). При μ = 1 ρ = 1, и клиппинг ни разу не срабатывает.

Где возник. Выбор μ = 1: в тексте честно пишем, что при этом клиппинг не работает и алгоритм — градиент политики с групповой базой и KL.

Где применяется. PPO и GRPO при μ > 1.

Усреднение loss по токенам и по ответам token-level vs sequence-level

Что это. Как сложить вклады токенов в одно число. Вклад токена: ℓ = −A·log π(токен); у всех токенов одного ответа A одинаковое. **По ответам**: сначала среднее внутри ответа (÷ длина ответа), потом среднее по ответам. **По токенам**: сумма по всем токенам шага ÷ общее число токенов.

Отличие. Штраф действительно за ответ целиком (одно A на ответ), но *в градиент* он попадает через каждый токен, и вопрос — какой вес получает один токен. Пример: два плохих ответа с A = −1, длиной 100 и 1000 токенов. По ответам каждый токен короткого получает вес 1/100, длинного — 1/1000: длинный плохой ответ на токен наказан в 10 раз слабее. Модель учится: «если не уверена — пиши длиннее», ответы раздуваются. По токенам — оба получают вес 1/1100 на токен: каждый плохой токен наказан одинаково. Обратная сторона: длинный хороший ответ тоже весит больше; у нас это эпизоды с поиском — смотрим в журнале.

Где возник. Обсуждение с Codex и вопрос 5 «про усреднение не поняла; штраф одинаковый и за длинное, и за короткое».

Где применяется. По ответам — исходный GRPO; по токенам — DAPO, Dr. GRPO. У нас — по токенам, знаменатель — число токенов модели во всём шаге.

4. KL и опорная модель

KL-дивергенция Kullback–Leibler divergence

Что это. Мера того, насколько одно распределение отличается от другого: $KL(P\|Q) = \Sigma P(x)\cdot\log(P(x)/Q(x)) \geq 0$, равна нулю только при P = Q. Несимметрична: $KL(P\|Q) \neq KL(Q\|P)$.

Отличие. «Обратный» $KL(\pi_\theta\|\pi_{ref})$ (в RL для языковых моделей) наказывает политику за токены, которые опорная модель считает маловероятными; «прямой» $KL(\pi_{ref}\|\pi_\theta)$ — за то, что политика перестала давать вероятность вариантам опорной.

Где возник. Штраф KL в GRPO и PPO: держит модель рядом с опорной, замедляя уход в лазейки награды.

Где применяется. RLHF (InstructGPT), GRPO, DPO (неявно), VAE, дистилляция.

Оценки KL по одному токену: k1, k2, k3

Что это. Точный KL требует суммы по всему словарю (150 тыс. токенов) на каждой позиции — дорого. Берут оценку по одному токену, который модель сгенерировала. Пусть d = log π_{ref}(токен) − log π_θ(токен). Тогда k1 = −d; k2 = $d^2/2$; k3 = e^d − d − 1.

Отличие. k1 — несмещённая, но бывает отрицательной и очень шумная. k2 — смещённая по значению, но в loss даёт градиент как раз обратного KL. k3 — несмещённая по значению и всегда ≥ 0 (поэтому хороша для журнала), но в loss её градиент в среднем равен градиенту *прямого* KL, а не того, что мы хотим. Отсюда решение: k2 в loss, k3 в журнал.

Где возник. Поправка Codex к формуле KL в GRPO (в DeepSeekMath — k3 в loss).

Где применяется. Schulman, «Approximating KL Divergence» (2020); GRPO (k3); работы 2025 г. о смещении градиента KL-штрафа.

Коэффициент β: фиксированный и адаптивный

Что это. Вес KL-штрафа: L = $L_{политики}$ + β·KL. **Фиксированный** — одно число на весь запуск (у нас 0,02). **Адаптивный** — после каждого шага β подстраивается, чтобы KL держался у заданной цели: KL выше цели → β растёт, ниже → падает.

Отличие. Адаптивный избавляет от подбора β, но требует подбора цели KL — то же самое, только в другой единице. Хуже для сравнения: у GRPO и PPO β менялся бы по-разному, и разница результатов смешалась бы с разницей регуляризации. И адаптивный β может «отпустить» модель, если KL долго ниже цели. Плюс адаптивного — не нужно угадывать масштаб KL заранее.

Где возник. Вопрос 6: «почему бы не взять адаптивный?». Предложение: фиксированный β в основных запусках (одинаковый для GRPO и PPO), адаптивный — как дополнительная абляция, если KL уйдёт слишком далеко. Ждёт решения.

Где применяется. Адаптивный — Ziegler и др. (2019), исходный PPO (вариант с KL-штрафом); фиксированный — InstructGPT, DeepSeekMath (β = 0,04).

KL к опорной модели и KL к предыдущей политике

Что это. KL к **опорной** — насколько модель ушла от исходной SFT-модели за всё обучение; это штраф в loss. KL к **предыдущей политике** — насколько модель изменилась за *один* шаг (в PPO — `approx_kl`); это только показатель в журнале.

Отличие. Первый растёт постепенно и ограничивается β; резкий скачок второго — признак слишком большого шага (lr велик или плохой пакет).

Где возник. Вопрос 12 о журнале обучения.

Где применяется. Оба пишут в журнал в PPO-реализациях (TRL, OpenRLHF); у нас — оба в журнал, первый ещё и в loss.

KL в VAE и KL в RL

Что это. В вариационном автокодировщике KL(q(z|x)‖N(0, I)) тянет распределение скрытого кода к стандартному нормальному — пространство кода становится гладким, из него можно сэмплировать.

Отличие. Формула та же, роль другая: в VAE KL сравнивает код с *фиксированным* априорным распределением, в RL — распределения токенов политики с *опорной моделью*. «Разреженность» — третья вещь: L1 или KL к малой вероятности активации в разреженных автокодировщиках.

Где возник. Ваш вопрос о связи с KL из VAE.

Где применяется. VAE (Kingma, Welling, 2014), β-VAE; в RL — см. выше.

Опорная модель reference model, π_{ref}

Что это. Замороженная модель, к которой привязан KL-штраф. У нас — SFT-модель: T-lite с влитым SFT-адаптером; в обучающем коде это те же веса с выключенным RL-адаптером (лишней копии модели в памяти нет).

Отличие. От π_{old} (модель, сгенерировавшая пакет): опорная не меняется весь запуск, π_{old} — новая каждый шаг.

Где возник. Пункт 1 плана GRPO: что считать опорной и с чего стартует RL.

Где применяется. InstructGPT, GRPO, DPO.

LoRA и слияние адаптера low-rank adaptation, merge

Что это. LoRA добавляет к матрице W поправку $(\alpha/r)\cdot B\cdot A$ малого ранга r и обучает только A и B (у нас r = 32, α = 32, все линейные слои). **Слияние** — один раз прибавить поправку к W и дальше работать с обычной моделью.

Отличие. Слитый адаптер нельзя «выключить» и он ничего не стоит при работе; неслитый можно выключать и менять. Поэтому SFT-адаптер вливаем (он больше не меняется), а RL-адаптер держим отдельно: он меняется каждый шаг, и vLLM подгружает его заново.

Где возник. SFT (адаптер `runs/sft/20261107_1913/adapter`, эпоха 3) и старт RL.

Где применяется. Hu и др. (2021); PEFT, vLLM (подгрузка адаптеров).

5. Оптимизатор и точность чисел

Скорость обучения и размер шага learning rate vs step size

Что это. **Скорость обучения** (lr) — настройка, множитель в формуле обновления. **Размер шага** — насколько на самом деле сдвинулись веса за одно обновление, $\|\Delta w\|$. В простом градиентном спуске $\Delta w = -\mathrm{lr}\cdot g$, то есть размер шага = lr × длина градиента: lr — «ручка», размер шага — результат ручки и градиента вместе.

Отличие. В учебниках «step size» часто употребляют как синоним lr — отсюда путаница. Но при одном lr шаг может быть большим (большой градиент) или маленьким. В Adam иначе: $\Delta w = -\mathrm{lr}\cdot\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$ — градиент делится на свою же среднюю величину, поэтому шаг по каждому весу примерно ≤ lr и почти не зависит от масштаба градиента. Отсечение градиента меняет именно размер шага, не трогая lr.

Где возник. Вопрос 11: «размер шага — это же learning rate». Почти: lr задаёт шаг, но шаг — это ещё и градиент.

Где применяется. У нас AdamW, lr = 1e-5 для RL (в SFT — свой lr с разогревом `warmup_steps`).

AdamW и его параметры betas, weight decay

Что это. Adam хранит скользящее среднее градиента (m, «инерция», β_1) и квадрата градиента (v, β_2) и делит одно на корень другого. AdamW — Adam с отдельным weight decay. У нас betas = (0,9; 0,95): β_2 = 0,95 вместо 0,999 — v быстрее реагирует на резкую смену масштаба градиента, что в RL с шумной наградой устойчивее.

Отличие. «β» здесь — не β из KL: это просто одинаковая буква.

Где возник. Настройки оптимизатора от Codex.

Где применяется. Почти все современные языковые модели; β_2 = 0,95 — GPT-3, LLaMA, многие RL-рецепты.

Weight decay

Что это. На каждом шаге веса немного стягиваются к нулю: $w \leftarrow w - lr \cdot \lambda \cdot w$. Мягкая защита от переобучения.

Отличие. От KL: weight decay тянет *веса* к нулю, KL — *поведение* к опорной модели. Для LoRA ноль весов адаптера = исходная модель, так что они похожи по смыслу, но KL меряет то, что важно (распределение ответов). Поэтому weight decay = 0.

Где возник. Настройки оптимизатора, вопрос 10 вашего списка.

Где применяется. Предобучение (обычно 0,1); в RL с LoRA чаще 0.

Отсечение градиента gradient clipping

Что это. Если длина вектора градиента больше порога (у нас 1,0), он уменьшается до длины порога; направление не меняется.

Отличие. От клиппинга PPO (ограничивает изменение вероятностей) и от нижней границы 0,02 в делителе преимущества (раздел 2). В Adam постоянное отсечение мало что меняет (масштаб и так нормируется), но разовый огромный градиент от неудачного пакета не испортит m и v.

Где возник. Настройки от Codex.

Где применяется. Почти всегда при обучении трансформеров (порог 1,0 — стандарт).

Масштабирование loss loss scaling, GradScaler

Что это. Во float16 числа меньше $\sim 6 \cdot 10^{-8}$ округляются в ноль, и мелкие градиенты пропадают. Поэтому loss перед обратным проходом умножают на большое S (например, 65 536), градиенты получаются в S раз больше и не теряются, а перед шагом оптимизатора делятся обратно на S. Если вышла бесконечность — шаг пропускается, S уменьшается.

Отличие. Это **не нормировка** — см. сводную таблицу в разделе 8: результат математически не меняется, только спасаются числа. Отсечение градиента — после деления на S.

Где возник. Настройки от Codex: V100 не поддерживает bfloat16, обучаем во float16.

Где применяется. Смешанная точность (Micikevicius и др., 2018), `torch.amp.GradScaler`.

float16 и bfloat16; compute capability

Что это. Оба формата — 16 бит. float16: 5 бит порядка, 10 бит мантиссы — точнее, но диапазон узкий (до 65 504). bfloat16: 8 бит порядка (как float32), 7 бит мантиссы — грубее, но не переполняется, масштабирование loss не нужно. **Compute capability** — номер поколения GPU NVIDIA, от него зависит, какие операции есть «в железе»: V100 — 7.0 (Volta), A100 — 8.0 (Ampere).

Отличие. bfloat16 и FlashAttention-2 требуют 8.0 и выше — на V100 их нет; отсюда float16, GradScaler, xformers и отсутствие кеширования префиксов в vLLM.

Где возник. Сервер TeraCloud: 2 × V100 32 ГБ; выбор vLLM 0.8.5 (последние версии с поддержкой Volta).

Где применяется. Выбор формата и ядер под конкретную карту.

6. Генерация

Температура, top-k, top-p sampling parameters

Что это. Как из распределения следующего токена выбирать токен. **Температура** T: логиты делятся на T; T < 1 — распределение острее (увереннее), T > 1 — площе. **top-k**: оставить k самых вероятных токенов, остальным — ноль, перенормировать. **top-p**: оставить самые вероятные токены, пока их суммарная вероятность не достигнет p. В `transformers` порядок: температура → top-k → top-p.

Отличие. Температура меняет форму всего распределения, top-k и top-p отрезают «хвост». top-k режет фиксированное число токенов, top-p — по суммарной вероятности (при уверенной модели остаётся 1–2 токена, при неуверенной — много).

Где возник. Находка 08.11: `generation_config` T-lite содержит T = 0,7, top_k = 20, top_p = 0,8; наш генератор переопределял T и top-p, но не top-k — EXP-010, EXP-012 и примеры ответов после SFT шли с top-k 20.

Где применяется. Любая генерация; в RL обычно T ≈ 1 и без отсечения хвоста, чтобы вероятности, по которым учимся, совпадали с теми, по которым генерировали, и ответы были разнообразнее.

generation_config

Что это. Файл `generation_config.json` рядом с весами модели: параметры генерации по умолчанию от авторов. `model.generate` берёт оттуда всё, что не задано явно.

Отличие. От параметров в вызове: явные аргументы переопределяют файл, но только те, что переданы. Отсюда «тихий» top-k 20.

Где возник. Проверка Codex и ваша проверка на сервере.

Где применяется. transformers; vLLM 0.8.5 тоже читает его по умолчанию (`generation_config="auto"`) — поэтому в RL параметры задаём явно в обоих движках.

Teacher forcing

Что это. Режим, в котором модели на вход подают *правильный* текст целиком, и она в каждой позиции предсказывает следующий токен; ничего не генерируется и ничего не выбирается случайно. Так считается loss при SFT и eval loss.

Отличие. От генерации: при генерации модель продолжает свой собственный текст, и ошибки накапливаются; температура, top-k и top-p влияют только на генерацию и никак — на teacher forcing.

Где возник. Ваш вопрос: «адаптер выбран с top-k — не перевыбрать ли?». Адаптер выбирали по eval loss, а он считается teacher forcing, поэтому top-k на выбор не влиял.

Где применяется. Обучение и оценка языковых моделей (loss, перплексия).

vLLM, continuous batching, PagedAttention, кеш KV

Что это. **vLLM** — движок генерации: отдельный процесс со своей копией весов. **Кеш KV** — сохранённые ключи и значения внимания всех прошлых токенов, чтобы не пересчитывать их на каждом новом токене (у T-lite — 147 КБ на токен: 36 слоёв × 8 голов × 128 × 2 (K и V) × 2 байта). **PagedAttention** — кеш KV хранится блоками (страницами), выделяемыми по мере роста текста, а не одним куском «под максимальную длину» на каждый эпизод. **Continuous batching** — новые эпизоды входят в пакет, как только старые закончились, а не ждут, пока закончится самый длинный.

Отличие. Выигрыш vLLM — не от отдельной карты (та просто убирает конкуренцию с обучением за память), а от этих двух приёмов: память под кеш не пропадает на резерв и дробление, поэтому одновременно помещается больше эпизодов, и карта не простаивает в ожидании длинного эпизода. Обычный `generate` резервирует кеш на весь пакет и ждёт самого длинного.

Где возник. Вопрос 7: «за счёт чего vLLM даёт выигрыш по памяти?»; схема 2 (vLLM на карте 1, обучение на карте 0). Кеширование префиксов (общий системный промпт считается один раз) на V100 в 0.8.5 не работает.

Где применяется. Kwon и др. (2023); генерация в TRL, OpenRLHF, veRL.

Prefill и decode

Что это. Две фазы генерации. **Prefill** — прочитать весь промпт за один проход и заполнить кеш KV (много токенов параллельно, быстро). **Decode** — дописывать по одному токену, каждый раз читая весь кеш (медленно, упирается в память).

Отличие. Длинный промпт дорог в prefill, длинный ответ — в decode. В эпизодах с поиском после каждого вызова результат инструмента дописывается — снова prefill.

Где возник. Оценка времени генерации эпизодов для RL.

Где применяется. Анализ скорости любых движков генерации.

Ядро внимания: eager, SDPA, xformers, FlashAttention

Что это. Внимание — операция $\mathrm{softmax}(QK^T/\sqrt{d})\cdot V$; **ядро** — программа для GPU, которая её считает. Результат одинаковый, память — разная. **eager** (наивное) хранит всю матрицу оценок «каждый токен с каждым» — L×L на голову. **Экономные** ядра (FlashAttention, память-экономное ядро xformers) считают матрицу кусками и никогда не держат её целиком: память растёт как L, а не L^2. **SDPA** — функция PyTorch, сама выбирающая ядро (flash — новые карты, memory-efficient — есть на V100, math — наивное).

Отличие — пример расчёта (вопрос 8). 32 головы, float16 (2 байта), один слой:

Длина L	Наивное: $L^2 \times 32 \times 2$ байта	xformers: Q, K, V, выход — $4 \times L \times 32 \times 128 \times 2$ байта
8 000	$8\,000^2 \times 64 \approx 4{,}1$ ГБ (только оценки; softmax — ещё столько же)	$4 \times 8\,000 \times 8\,192 \approx 0{,}26$ ГБ
16 384	$16\,384^2 \times 64 \approx 17{,}2$ ГБ — не помещается	$4 \times 16\,384 \times 8\,192 \approx 0{,}54$ ГБ

Время у обоих ~L^2, но экономное быстрее за счёт того, что не гоняет огромную матрицу через память.

Где возник. vLLM на V100 (бэкенд xformers, `VLLM_USE_V1=0`); SFT на 12 тыс. токенов уместился в 24,8 ГБ — значит, работало экономное ядро.

Где применяется. FlashAttention (Dao и др., 2022), xformers (Meta), Rabe и Staats (2021) — «внимание без $O(n^2)$ памяти».

Расхождение log p между vLLM и обучающим кодом

Что это. vLLM при генерации возвращает log p выбранных токенов; обучающий код считает их заново через `transformers`. Веса одни, но программы разные (ядра, порядок суммирования во float16): −0,105 против −0,112 — обычный шум.

Отличие. Шум безвреден; опасно систематическое расхождение (другая температура или top-k в одном из движков, ошибка подгрузки LoRA). Поэтому считаем дважды: vLLM — бесплатный побочный продукт для проверки, обучающий проход — *обязательный*: только он даёт градиент (у vLLM log p — просто числа без связи с весами).

Где возник. Вопрос 12: «считаем вероятности два раза — зачем?».

Где применяется. Все связки «отдельный движок генерации + обучение» (veRL, OpenRLHF) пишут это расхождение в журнал.

7. Оценка

Eval loss (loss на отложенных данных)

Что это. Средняя кросс-энтропия на примерах, не участвовавших в обучении, при teacher forcing; у нас — только по токенам реплик модели.

Отличие. От train loss: тот падает всегда, eval loss начинает расти при переобучении. От качества генерации: низкий loss значит «модель хорошо предсказывает тексты учителей», а не «хорошо пишет сама».

Где возник. Выбор адаптера SFT: прогон на 7 эпох, лучший — эпоха 3 (1,417).

Где применяется. Ранняя остановка и выбор чекпоинта в SFT.

Перплексия perplexity

Что это. exp(средний loss на токен) — «среди скольких равновероятных вариантов модель в среднем выбирает». eval loss 1,417 → перплексия $e^{1{,}417} \approx 4{,}1$.

Отличие. Та же информация, что loss, в другой шкале. Сравнима только при одном токенизаторе и одних текстах.

Где возник. Проверка SFT: перплексия на общих текстах до и после — не разучилась ли модель обычному языку.

Где применяется. Оценка языковых моделей; проверка забывания.

ROC AUC и доля верно упорядоченных пар

Что это. ROC AUC — вероятность того, что случайный «хороший» пример получит оценку выше случайного «плохого» (0,5 — угадывание, 1 — идеал). Доля верно упорядоченных пар — то же самое, но по заранее заданным парам «оригинал — его испорченная версия».

Отличие. AUC сравнивает всех хороших со всеми плохими (в том числе с разных тем — тут мешает разная сложность тем); пары сравнивают внутри задания — ближе к тому, как работает GRPO. Корректность зависит от того, откуда «правильный порядок»: в синтетических парах он объективен (испортили сами), в парах с конфликтом достоинств — субъективен (один разметчик, вы), это надо оговорить в тексте.

Где возник. Вопрос 9: «насколько это корректно, моя оценка ведь тоже субъективна; только на итоговой награде? а график?». AUC — свойство награды, а не модели: считаем до и после изменений награды. «Согласие награды с судьёй по ходу RL» — уже график, предлагаю отдельно (ждёт решения).

Где применяется. Оценка моделей награды (RewardBench — доля верно упорядоченных пар), классификаторы.

LLM-судья

Что это. Большая языковая модель, которой дают два ответа (или один) и критерии и просят оценить. Вслепую — не говоря, какая модель что писала; в двух порядках — чтобы убрать предпочтение первого или второго.

Отличие. От награды: судья дорогой и медленный, поэтому не для обучения, а для независимой проверки. Не должен быть той же модели-основы, что и обучаемая (поэтому вопрос, не на Qwen ли GigaChat).

Где возник. Проверка SFT и выбор чекпоинта RL; кандидаты — GigaChat-3-Ultra, DeepSeek V4.

Где применяется. MT-Bench, AlpacaEval (Zheng и др., 2023).

distinct-n

Что это. Доля уникальных n-грамм (последовательностей из n слов) среди всех n-грамм в наборе ответов. Мера разнообразия.

Отличие. Мерит повторяемость текста, а не качество. Падение после SFT или RL — признак «коллапса» к шаблонным ответам.

Где возник. Функция `diversity` в `sft_eval.ipynb`.

Где применяется. Li и др. (2016); оценка диалоговых моделей.

Забывание catastrophic forgetting

Что это. Дообученная на узкой задаче модель хуже справляется с тем, что умела раньше.

Отличие. От переобучения: переобучение — хуже на новых примерах *той же* задачи, забывание — хуже на *других* задачах.

Где возник. Проверка SFT (перплексия на общих текстах); LoRA и KL как защита от него.

Где применяется. Любое дообучение.

Выбор чекпоинта: плато и вето на лазейки

Что это. Чекпоинты сохраняются каждые 10 шагов. Короткий список — по награде на validation; из него **исключаются** (вето) чекпоинты с признаками лазеек (рост доли слайдов «почти одни ключи», падение доли эпизодов с поиском и т. п.); из оставшихся выбирает слепой LLM-судья. Остановка — на плато независимой метрики.

Отличие. От выбора по обучающей награде: та растёт и за счёт лазеек, выбрали бы модель, лучше всех ими пользующуюся.

Где возник. Вопросы 10 и 12: «отбираем по нашей метрике, но не учитывая награду лазеек?» — да: награда по-прежнему главная, но чекпоинт с признаками лазейки не может победить, какой бы высокой ни была награда.

Где применяется. Gao и др. (2023), «Scaling Laws for Reward Model Overoptimization».

8. Сводные сравнения

Масштабирование и нормировка

Масштабирование — умножение на заранее выбранное число; форма и относительные расстояния не меняются, часто масштаб потом убирают обратно. **Нормировка** — приведение к общей шкале *по самим данным* (вычесть среднее, разделить на разброс, разделить на сумму); множитель зависит от данных и меняет то, чему учится модель.

Пример	Что делает	Тип	Меняет ли результат
GradScaler (loss × S, градиент ÷ S)	спасает мелкие числа во float16	масштабирование	нет (только численно)
Веса компонентов награды (ключи 0,5, визуал 0,2, λ = 1)	задаёт важность компонента	масштабирование	да, это выбор приоритетов
(α/r) в LoRA	фиксированный множитель поправки	масштабирование	эквивалентно смене lr
A = (r − среднее группы) / std шага	убирает уровень задания, приводит к общей единице	нормировка	да
Усреднение loss по токенам (÷ число токенов шага)	делает loss не зависящим от размера пакета	нормировка	да (вес длинных и коротких ответов)
Пороги m = 0,25σ, ε = 0,1σ в проверке награды	выражают эффект в разбросе внутри задания	нормировка	да (критерий принятия)
Softmax, top-p с перенормировкой	делает сумму вероятностей равной 1	нормировка	да
RMSNorm в слоях модели	приводит активации к единичной средней норме	нормировка (с обучаемым масштабом)	да

Размер шага и скорость обучения

	Скорость обучения (lr)	Размер шага ‖Δw‖
Что это	настройка, задаётся заранее (1e-5)	фактическое изменение весов за обновление
SGD	множитель	lr × ‖g‖ — зависит от градиента
Adam	множитель	≈ lr на вес, почти не зависит от масштаба градиента
Что на него влияет	расписание (разогрев, спад)	lr, градиент, отсечение градиента, нормировка преимущества
Как следим	пишем в журнал	KL к предыдущей политике, норма градиента

Три «клиппинга»

	Ограничивает	Где	У нас
Клиппинг PPO	изменение вероятности токена (ρ в [0,8; 1,2])	функция цели	при μ = 1 не срабатывает
Отсечение градиента	длину вектора градиента (≤ 1,0)	после обратного прохода	да
Нижняя граница делителя	std шага снизу (≥ 0,02)	нормировка преимущества	да

Три KL

	Между чем	Роль
KL к опорной модели	π_θ и SFT-модель	штраф в loss (β = 0,02, k2) + журнал (k3)
KL к предыдущей политике	π_θ до и после шага	журнал: признак слишком большого шага
KL в VAE	код q(z|x) и N(0, I)	регуляризация пространства кода — для сравнения

Составлено 08.11.2026 для дипломной работы «Исследование агентных архитектур для генерации презентаций».